

## DOMAIN 3 — Data Retrieval

Domain 3 focuses on retrieving, filtering, analyzing, and organizing data using SQL queries. This domain is heavily tested in the Certiport Database certification exam because it measures whether students truly understand how databases work in real situations.

Many students struggle in Domain 3 because the exam questions are not simple memorization questions. Instead, they require:

- analyzing SQL queries
- identifying the best query
- understanding query behavior
- recognizing filtering logic
- distinguishing similar SQL clauses

This reviewer focuses not only on definitions, but also on:

- SQL behavior
- common exam traps
- troubleshooting
- scenario analysis
- query interpretation

## UNDERSTANDING QUERY FLOW

One of the biggest mistakes students make is not understanding how SQL queries logically execute.

## SQL QUERY FLOW

Even though queries are written in this order:

1. SELECT
2. FROM
3. WHERE
4. GROUP BY
5. HAVING
6. ORDER BY

SQL logically processes them like this:

| Logical Order | Clause |
|---------------|--------|
| 1             | FROM   |
| 2             | JOIN   |

|   |          |
|---|----------|
| 3 | WHERE    |
| 4 | GROUP BY |
| 5 | HAVING   |
| 6 | SELECT   |
| 7 | ORDER BY |

Understanding this helps students answer:

- troubleshooting questions
- aggregation questions
- filtering questions

## SELECT STATEMENT

The SELECT statement retrieves data from a database table.

BASIC QUERY

```
SELECT *  
FROM Students;
```

This retrieves all columns from the Students table.

CERTIPORT TRAP

Students often think:

SELECT \* is always acceptable.

Reality:

- It may reduce performance.
- It retrieves unnecessary columns.
- In real systems, specific columns are preferred.

BETTER PRACTICE

```
SELECT StudentID, LastName  
FROM Students;
```

## INNER JOIN

INNER JOIN returns ONLY matching records between tables.

## EXAMPLE

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
INNER JOIN Orders
ON Customers.CustomerID = Orders.CustomerID;
```

## WHAT HAPPENS?

- Only customers with orders appear.
- Customers without orders are excluded.

## CERTIPOINT SCENARIO

Scenario: A company wants a report showing **ONLY** customers who purchased products.

Correct Answer: INNER JOIN

## COMMON TRAP

Students confuse:

INNER JOIN  
LEFT JOIN

Remember:

|            |                       |
|------------|-----------------------|
| JOIN       | Result                |
| INNER JOIN | Matching only         |
| LEFT JOIN  | Keep all left records |

## LEFT JOIN

LEFT JOIN preserves ALL records from the left table.

## EXAMPLE

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
LEFT JOIN Orders
ON Customers.CustomerID = Orders.CustomerID;
```

## WHAT HAPPENS?

- All customers appear.
- Customers without orders show NULL values.

## CERTIPORT SCENARIO

Scenario: A company wants to display ALL customers, including customers who never placed orders.

Correct Answer: LEFT JOIN

## COMMON TRAP

Students choose INNER JOIN because they see “orders” in the question. Focus carefully on:

- “including customers with NO orders”
- That phrase strongly suggests LEFT JOIN.

## RIGHT JOIN

RIGHT JOIN preserves all records from the right table. Less commonly used but still appears in certification exams.

## FULL OUTER JOIN

FULL OUTER JOIN returns:

- matching records
- unmatched left records
- unmatched right records

## SELF JOIN

A SELF JOIN joins a table to itself.

## REAL-WORLD EXAMPLE

Employee table:

| EmployeeID | EmployeeName | ManagerID |
|------------|--------------|-----------|
| 1          | Bryan        | NULL      |
| 2          | Anna         | 1         |

The same table stores:

- employees
- Managers

A SELF JOIN connects employees to managers.

## CARTESIAN PRODUCT

A Cartesian Product occurs when tables are joined without proper conditions.

EXAMPLE

```
SELECT *  
FROM Customers, Orders;
```

Without a JOIN condition:

- every customer combines with every order
- This creates massive incorrect results.

CERTIPOINT TRAP

- Students may forget the ON condition.
- That usually creates a Cartesian Product.

## DISTINCT

DISTINCT removes duplicate values.

EXAMPLE

```
SELECT DISTINCT City  
FROM Customers;
```

WHAT HAPPENS?

Duplicate cities are removed.

COMMON TRAP

DISTINCT does NOT:

- remove duplicate rows entirely
- modify the table permanently
- It only affects query results.

## UNION

UNION combines result sets vertically.

### EXAMPLE

```
SELECT City FROM Customers
UNION
SELECT City FROM Suppliers;
```

### IMPORTANT RULES

- ✓ Same number of columns
- ✓ Compatible data types
- ✓ Similar structure

### UNION VS JOIN

|                          |                              |
|--------------------------|------------------------------|
| UNION                    | JOIN                         |
| Combines rows vertically | Combines tables horizontally |
| Requires same columns    | Uses relationships           |

### CERTIPOINT TRAP

Students confuse:

- UNION
- JOIN

Remember:

- UNION stacks results
- JOIN merges related columns

## WHERE CLAUSE

WHERE filters rows BEFORE grouping.

### EXAMPLE

```
SELECT *
FROM Employees
WHERE Salary > 50000;
```

#### CERTIPOINT TRAP

- WHERE cannot filter aggregate results.
- This causes confusion with HAVING.

### GROUP BY

GROUP BY groups records sharing common values.

#### EXAMPLE

```
SELECT City, COUNT(*)  
FROM Customers  
GROUP BY City;
```

#### WHAT HAPPENS?

Records are grouped by City.

### HAVING

HAVING filters grouped results AFTER aggregation.

#### EXAMPLE

```
SELECT City, COUNT(*)  
FROM Customers  
GROUP BY City  
HAVING COUNT(*) > 5;
```

#### WHY NOT WHERE?

Because:

- WHERE filters rows
- HAVING filters groups

#### CERTIPOINT TRAP

This is one of the MOST COMMON exam traps.

Students frequently misuse:

- WHERE
- HAVING

## EASY MEMORY TECHNIQUE

| Clause | Filters         |
|--------|-----------------|
| WHERE  | Individual rows |
| HAVING | Groups          |

## ORDER BY

ORDER BY sorts query results.

ASCENDING ORDER  
ORDER BY LastName ASC;

DESCENDING ORDER  
ORDER BY Salary DESC;

CERTIPORT TRAP

- Students forget:
- ASC is default
  - DESC means highest to lowest

## LIKE OPERATOR

LIKE searches patterns using wildcards.

WILDCARDS

| Symbol | Meaning                  |
|--------|--------------------------|
| %      | Any number of characters |
| _      | Single character         |

EXAMPLES

WHERE LastName LIKE 'A%'

Starts with A.

WHERE LastName LIKE '%son'

Ends with "son".



## COMMON TRAP

Students confuse:

- %
- \_

Remember:

% = many characters  
\_ = one character only

## BETWEEN

BETWEEN retrieves values within a range.

EXAMPLE

WHERE Age BETWEEN 18 AND 25

IMPORTANT NOTE

BETWEEN is inclusive.

Meaning:

18 included  
25 included

## IN OPERATOR

IN searches multiple values efficiently.

EXAMPLE

WHERE City IN ('Manila', 'Cebu')

WHY USE IN?

Cleaner than:

WHERE City='Manila'  
OR City='Cebu'

## NULL VALUES

NULL means:

- missing
- unknown
- Empty

NULL is NOT:

- zero
- blank text

CORRECT NULL SEARCH  
WHERE Email IS NULL

WRONG WAY  
WHERE Email = NULL

This is incorrect SQL logic.

CERTIPORT TRAP

Students frequently forget:

IS NULL  
IS NOT NULL

## AGGREGATE FUNCTIONS

Aggregate functions summarize data.

- COUNT() - Counts records.

```
SELECT COUNT(*)  
FROM Orders;
```

- SUM() - Adds numeric values.
- AVG() - Calculates average.
- MIN() and MAX() - Return smallest and largest values.